

Clawdbot LLM Run Engine Architecture

Entry Point

runEmbeddedPiAgent()

- Lane Queueing
- Workspace Resolution
- Config Loading
- Model Registry Init

Model & Auth Resolution

- resolveModel()
- Context Window Guard
 - Warn: 8k tokens
 - Min: 2k tokens
- Auth Profile Store
- Profile Order
- Cooldown Tracking

Main Execution Loop

1. runEmbeddedAttempt()
2. Success? → Build Payloads

3. Error Classification:

- ContextOverflow → Compact Session
- AuthError / RateLimit → Rotate Profile
- FailoverError → Model Fallback

4. Retry or Throw

Result

EmbeddedPiRunResult

- Payloads • Meta • Usage

Attempt Execution

runEmbeddedAttempt()

Initialization:

- Session Write Lock
- Load Session Manager
- Build System Prompt
- Load Workspace Skills
- Resolve Sandbox Context

History Processing:

- Limit Turns (DM:100, else:30)
- Sanitize for Provider
- Validate Turn Order
- Inject History Images

Streaming & Execution:

Session Compaction

- Create Agent Session
- Set Thinking Level
- Stream LLM Response
- Execute Tool Calls
- Compact EmbeddedPiSessionDirect()
- Save Session State

Triggered on Context Overflow:

- Load Session History
- Calculate Token Budget
- Generate Summary via LLM
- Replace Old Turns

Error Classification & Handlers

- Save Compacted Session
- Return Compaction Report

Error Types:

- ContextOverflow → Compact
- AuthError → Rotate Profile
- RateLimitError → Cooldown
- TimeoutError → Fallback
- CompactionFailure → Failover
- FailoverError → Model Fallback

Failover Reasons:
auth | rate_limit | timeout
context_overflow | compaction_failure
role_ordering | unknown

Key Features

1. Auth Profile Rotation

- Multi-key failover on rate limit

2. Context Window Management

- Guard rails for insufficient context

3. Automatic Compaction

- Summarize old history on overflow

4. Thinking Level Fallback

- Reduce complexity on timeout

5. Sandbox Integration

- Isolated tool execution environments

6. Lane Queueing

- Session & global concurrency control

File Organization

7. Provider-Specific Handling

- Google, Anthropic, Azure adaptations

8. Cache Management

- EmbeddedPiCache with TTL timestamps

├─ run.ts

- | Main orchestration & loop
- ├─ run/
 - | └─ attempt.ts
 - | | Single attempt execution
 - | └─ params.ts
 - | | Parameter types
 - | └─ payloads.ts
 - | | Result building
 - | └─ images.ts
 - | | Image loading & injection
 - └─ types.ts
 - | Attempt-specific types
- ├─ compact.ts
 - | Session compaction logic
- ├─ model.ts
 - | Model resolution
- ├─ lanes.ts
 - | Queue lane management
- ├─ abort.ts
 - | Abort signal handling
- ├─ google.ts
 - | Google-specific adaptations
- └─ types.ts
 - Core type definitions

Auth Profile Rotation Flow

```
profileCandidates = [id1, id2, ...]
profileIndex = 0

while index < candidates.length:
    candidate = candidates[index]

    if inCooldown(candidate):
        index++
        continue

    try:
        apiKey = resolve(candidate)
        authStorage.setKey(apiKey)
        break
    catch:
        advanceAuthProfile()

On Success:
    markAuthProfileGood(profile)
```

Data Flow Summary

```
On Error:
    markAuthProfileFailure(profile)
    Start Request Cooldown period
    ↓
runEmbeddedPiAgent(params)
    ↓
[Model Resolution]
    ↓
[Auth Profile Selection]
    ↓
┌──────────────────┐
│ Main Loop      │
│ → Attempt      │
│ → Check Success │
│ → Handle Errors │
│ → Retry/Fallback│
└──────────────────┘
    ↓
Build Payloads
    ↓
EmbeddedPiRunResult
    → payloads[]
    → meta (usage, duration)
    → messagingToolSends
    → stopReason
```